

Вот полное, структурированное и готовое к копированию **Техническое Задание (Prompt)**.

Вы можете скопировать этот текст целиком и отправить его в **ChatGPT (o1/4o)**, **Claude 3.5 Sonnet** или в IDE с поддержкой AI (Cursor, Windsurf). Это ТЗ написано на языке разработчиков, чтобы нейросеть сразу выдала рабочий код.

SYSTEM PROMPT (Контекст для ИИ)

Ты — Senior Full Stack Developer. Твоя задача — реализовать Telegram Mini App для трекинга тренировок. Стек: Python (aiogram), Google Sheets (gsread), HTML/JS (Frontend). Ты должен написать код, следуя архитектуре ниже.

ТЕХНИЧЕСКОЕ ЗАДАНИЕ: Gym Logger Mini App

1. Описание проекта

Telegram-бот с веб-интерфейсом (Mini App) для записи подходов в тренажерном зале.

Ключевые особенности:

1. Использование Google Sheets как базы данных.
2. Интерфейс ввода данных через "барабаны" (iOS style pickers).
3. Автоматическая подстановка весов с прошлой тренировки.
4. Поддержка суперсетов (запись нескольких упражнений пакетом).
5. Отображение фото тренажера в чате перед вводом.

2. Стек технологий

- **Backend:** Python 3.10+, aiogram 3.x, gsread, oauth2client.
 - **Frontend:** HTML5, CSS3, Vanilla JS.
 - **UI Library:** mobile-select.js (для реализации барабанов выбора).
 - **Database:** Google Spreadsheet.
-

3. Структура Базы Данных (Google Sheets)

Необходимо работать с таблицей, содержащей 3 листа.

Лист 1: LOG (Журнал)

Хранит историю. Колонки:

- A Date (dd.mm.yyyy hh:mm)
- B Exercise (String)
- C Weight (Float)
- D Reps (Integer)
- E Rest (Integer, seconds)

- F Set_Group_ID (String/UUID) — для группировки суперсетов.

Лист 2: EXERCISES (Справочник)

- A Exercise Name
- B Muscle Group (Спина, Грудь, Ноги...)
- C Photo_File_ID (Telegram file_id фото тренажера)

Лист 3: LAST_RESULTS (Кэш)

Содержит последние результаты по каждому упражнению. Бот читает данные отсюда для формирования URL.

- A Exercise Name
- B Last Weight
- C Last Reps

(Предполагается, что эти данные обновляются формулами массива внутри Google Sheets, бот их только читает).

4. Функционал Backend (Python)

4.1. Обработка команд

- /start — Выводит меню с группами мышц (читает уникальные значения из EXERCISES!B:B).
- /add_exercise — FSM сценарий для добавления нового упражнения (Название -> Группа -> Фото). Сохраняет в лист EXERCISES.

4.2. Логика выбора упражнения

1. Пользователь выбирает Группу -> Бот выдает список упражнений (Inline Buttons).
2. Пользователь выбирает Упражнение.
3. **Действие Бота:**
 - Находит упражнение в листе EXERCISES, берет Photo_File_ID.
 - Находит последние результаты в листе LAST_RESULTS (если нет — дефолт 0).
 - Отправляет сообщение с фото (если есть).
 - Отправляет Inline-кнопку "**Записать подход**", открывающую WebApp.
 - URL WebApp генерируется с параметрами:

`https://my-site.com/?ex={Name}&w={LastWeight}&r={LastReps}`

4.3. Обработка данных от WebApp

Бот принимает JSON через web_app_data.

Структура входящего JSON:

JSON

```
{
  "type": "workout_data",
  "payload": [
    { "exercise": "Жим лежа", "weight": 100, "reps": 5, "rest": 120 },
    { "exercise": "Разводка", "weight": 20, "reps": 12, "rest": 60 }
  ]
}
```

Логика сохранения:

1. Генерирует уникальный UUID для этого пакета данных.
 2. Получает текущее время сервера.
 3. Проходит циклом по массиву payload.
 4. Добавляет строки в лист LOG: [Timestamp, Exercise, Weight, Reps, Rest, UUID].
 5. Отправляет пользователю подтверждение: "  Записано 2 упражнения".
-

5. Функционал Frontend (HTML/JS)

5.1. Интерфейс

- Чистый, минималистичный дизайн (Apple style).
- **Компоненты:**
 - Заголовок с названием текущего упражнения.
 - Текст "Прошлый раз: {w}кг x {r}".
 - **3 Барабана (Pickers):**
 1. **Вес:** Шаг 1.25 или 2.5. (Диапазон 0–300).
 2. **Повторы:** Шаг 1. (Диапазон 1–100).
 3. **Отдых:** Значения ["0 сек", "30 сек", "60 сек", "90 сек", "2 мин", ... "5 мин"]. Значение отправляется как число (секунды).
 - Кнопка "**Добавить в сет (+)**" (Secondary style).
 - Кнопка "**Сохранить и закрыть**" (Primary style, большая, внизу).

5.2. Логика JavaScript

1. **Инициализация:**
 - Парсинг URL Query Params (ex, w, r).
 - Инициализация MobileSelect (барабанов) с предустановкой значений из URL.
2. **Буфер данных:**
 - Создать массив workoutBuffer = [].

3. Кнопка "Добавить в сет (+)":

- Считывает текущие значения барабанов.
- Добавляет объект в workoutBuffer.
- Визуально показывает бейдж "В буфере: N упр."
- Сбрасывает UI или предлагает выбрать следующее упражнение (для MVP можно просто очистить поля и позволить изменить название, либо просто накапливать сеты одного упражнения, если это дроп-сет. **Уточнение:** Для суперсетов разных упражнений в MVP версии просто меняем название упражнения в заголовке на "Упражнение 2" или реализуем простой input select).
- *Упрощение для MVP:* Кнопка "+" просто дублирует текущее упражнение в буфер (для дроп-сетов) или позволяет сохранить текущее и не закрывать окно.

4. Кнопка "Сохранить":

- Добавляет текущие значения в workoutBuffer (если они еще не там).
- Формирует итоговый JSON.
- Вызывает tg.sendData(JSON.stringify(finalObj)).
- Вызывает tg.close().

6. Инструкция для ИИ по генерации кода

Пожалуйста, сгенерируй код в следующей последовательности:

1. **requirements.txt** (список библиотек).
2. **google_sheets.py** (модуль-обертка для работы с gspread: чтение кэша, запись лога, чтение упражнений).
3. **bot.py** (основной файл бота: хендлеры команд, генерация ссылки WebApp, обработка входящих данных).
4. **index.html** (полный код фронтенда в одном файле: HTML + CSS + JS логика + подключение CDN библиотек).

Код должен быть снабжен комментариями. Учти обработку ошибок (например, если Google API недоступен).